

Rockchip Developer Guide Android 3A Module

文件标识: RK-KF-YF-793

发布版本: V1.0.3

日期: 2025-04-19

文件密级: ☐绝密 ☐秘密 ☐内部资料 ☒公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2024 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址: 福建省福州市铜盘路软件园A区18号

网址: www.rock-chips.com

客户服务电话: +86-4007-700-590

客户服务传真: +86-591-83951833

客户服务邮箱: fae@rock-chips.com

前言

概述

本文主要介绍Rockchip Android 3A模块，开发或技术支持人员通过阅读此文档，对Android平台3A模块有一个初步的了解，帮助阅读者开发和调试。

产品版本

芯片名称	内核版本
------	------

芯片名称	内核版本
适配所有芯片	适配所有版本

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	张贵朋	2024-06-19	初始版本
V1.0.1	张贵朋	2024-12-31	配置方式变更，补充授权说明
V1.0.2	张贵朋	2025-03-12	补充验证和 SAI 回环说明
V1.0.3	蓝斌元	2025-04-19	补充SAI、DLP及授权说明

目录

Rockchip Developer Guide Android 3A Module

- 说明
- 编译
 - 确认平台配置路径
 - 添加编译开关
- 启用
 - 算法库
 - 启用模块
 - 修改声道参数
 - 授权
 - Android
 - 声明音效
 - 绑定音效
 - 输入包含 MIC 数据和回采数据
 - MIC 数据和回采数据同源
 - MIC 数据和回采数据异源
 - 输入仅包含 MIC 数据
 - 调整音频参数

- 修改 MIC 录音参数
 - 关闭 libanr 和 libspeex
- 验证
 - 确认音效是否已经注册到 audioflinger
 - 确认音效是否能正常绑定到对应的 stream
 - 确认算法输出是否符合预期
- 算法参数
 - 算法参数说明
 - 算法参数调试
 - 更新算法参数
 - 分析算法输出
- Q&A
 - 如何确认需要启用哪些算法模块
 - 没有生成输出文件
 - 输出文件数据全 0
 - 加入 DLP 后, 无法以指定的声道 (MIC + 回采) 录音, 提示参数错误
 - 检查通路设置
 - 检查声卡相关

说明

阅读本文档时请注意标注的红色字体之处, 切勿遗漏

本音效库基于 Android Audio Effect 框架集成 librkaudio_vqe 算法库, 将音效库命名为 rkpreprocess

算法库分为通用版本和付费版本, SDK 初始版本为通用版本, 如果需要使用付费版本请联系我司业务申请授权

模块	描述
AEC	包括 delay
BF	包括 fast_aec aes eq gsc (仅适用于多 MIC) agc anr nlp dereverb cng dtd howl doa (仅适用于多 MIC) wind wakeup ainr (授权付费)

	描述
MIC数量	单 MIC 双 MIC 多 MIC (授权定制付费)

编译

确认平台配置路径

项目的 BoardConfig.mk 路径与芯片型号和 lunch 时选择的 product 有关

比如芯片名称为 rk3588, product 为 rk3588_u

```
# echo $TARGET_PRODUCT
# rk3588_u
```

则 BoardConfig.mk 的位置为 `device/rockchip/rk3588/rk3588_u/BoardConfig.mk`

添加编译开关

在 BoardConfig.mk 添加下面的编译开关, 确保 `hardware/rockchip/audio/preprocess` 被编译

```
BOARD_USE_AUDIO_PREPROCESS := true
```

启用

算法库

算法库详细的介绍请参考[文档](#)

启用模块

参考[如何确认需要启用哪些算法模块](#)

根据实际需求和当前算法库版本支持的模块, 修改 `hardware/rockchip/audio/preprocess/rkaudio_effect_preprocess.json` 中对应模块的 `status` 为 `enable` 来启用模块

模块的开关状态说明如下:

status	说明
enable	启用
disable	关闭

模块的依赖关系说明如下:

模块	依赖模块
aec	无
delay	aec
bf	无
fast_aec, aes, eq, gsc, agc, anr, nlp, dereverb, cng, dtd, howl, doa, wind, wakeup, ainr	bf

模块之间存在依赖关系时, 需要先启用依赖的模块才能启用某个模块

修改声道参数

修改 `hardware/rockchip/audio/preprocess/libs/include/rkaudio_preprocess.h` 的声道参数, 如下:

参数	说明
NUM_CHANNEL 宏	总的声道数, 比如 2 MIC + 2 回采, 则为 4
NUM_REF_CHANNEL 宏	回采声道数, 比如 2 回采, 则为 2
static short Array[NUM_CHANNEL]	声道排序, 从 0 开始, 比如 2 MIC + 2 回采, 如果没有乱序则为 {0, 1, 2, 3}

授权

如果当前使用的算法库是付费版本, 一般申请通过后, 回复邮件中会包含但不限于以下的授权信息:

	说明
用户	当前用户, 请勿外传
密码	密码, 请勿外传
次数	可使用次数
授权模块	audio3a

请参考[文档](#)使用授权工具进行授权, 如果是设备上直接授权**必须**保证设备可以联网, 如果是 PC 上授权**必须**保证 PC 可以联网

注意: 使用 `rkauth_tool_bin` 和 `rkauth_verify_licence` 时, 需要**追加**选项 `-m "audio3a"`

授权成功后将生成的授权信息文件 `key.lic` **保存**到 `/data/key.lic`

参考命令:

```
rkauth_tool_bin --user="你的账号" --passwd="你的密码" -m "audio3a" --output="/data/key.lic"
```

量产时, 需要对授权流程代码化, 可以根据授权工具及文档提供的方法, 自行完成, 也可以redmine提交需求协助

audio3a算法默认是访问的`/data/key.lic`, 如需变更key路径, 算法有如下接口:

```
int rkaudio_license_path_set(char* path);
```

`hardware/rockchip/audio/preprocess` 在 `rkaudio_preprocess.h` 中增加此函数声明并在 `algo_init` 函数开头调一下, 变更key的访问路径

Android

声明音效

需要在 audiopolicy 中 声明 rkpreprocess library 和 effect

修改 frameworks/av/media/libeffects/data/audio_effects.xml , 声明如下 library 和 effect:

```
<!-- libraries 已经声明，在原来的基础上追加下面的 library 项即可（这是一段注释，不用粘贴） -->
<libraries>
    <!-- 保留已有内容，添加下面的配置项（这是一段注释，不用粘贴） -->
    <library name="rkpreprocess" path="librkpreprocess.so"/>
</libraries>

<!-- effects 已经声明，在原来的基础上追加下面的 effect 项即可（这是一段注释，不用粘贴） -->
<effects>
    <!-- 保留已有内容，添加下面的配置项（这是一段注释，不用粘贴） -->
    <effect name="preproc_without_ref" library="rkpreprocess" uuid="ca002f38-5304-4e93-93de-0002a5d5c51b"/>
    <effect name="preproc_with_ref_interleaved" library="rkpreprocess"
uuid="bc23650c-dddf-47d6-a426-0002a5d5c51b"/>
    <effect name="preproc_with_ref_separated" library="rkpreprocess" uuid="d3d81983-b1fd-459f-99f4-0002a5d5c51b"/>
    <effect name="preproc_ref_cache" library="rkpreprocess" uuid="9adba6e4-ce07-4c50-afbf-0002a5d5c51b"/>
</effects>
```

直接复制可能存在自动换行, 请在复制后修正

effect 的说明如下:

effect	说明
preproc_without_ref	输入只有 MIC 数据, 不包含回采数据. 附着在对应的 input stream 上, 此时 rkpreprocess 只处理 MIC 数据.
preproc_with_ref_interleaved	输入包含 MIC 数据和回采数据, MIC 数据和回采数据来源于同一个声卡. 附着在对应的 input stream 上, 此时 rkpreprocess 同步处理 MIC 数据和回采数据.
preproc_with_ref_separated	输入包含 MIC 数据和回采数据, 但 MIC 数据来源于一个声卡, 回采数据来源于另一个声卡或 buffer. 附着在对应的 input stream 上, 此时 rkpreprocess 异步处理 MIC 数据和回采数据.
preproc_ref_cache	CPU 缓存回采数据, 作为软回采. PREPROC_ENABLE_ALOOP 开启时回采数据缓存到 Aloop 声卡, 关闭时回采数据缓存到 buffer. 附着在对应的 output stream 上.

effect 之间为互斥, 后续绑定音效的时候只能绑定以上一种

绑定音效

根据输入的数据构成, 绑定对应的 effect 到相应的 stream

输入包含 MIC 数据和回采数据

回采数据应用于 aec/fast_aec/aes 模块

MIC 数据和回采数据同源

同源是指 MIC 数据和回采数据来源于同一个声卡

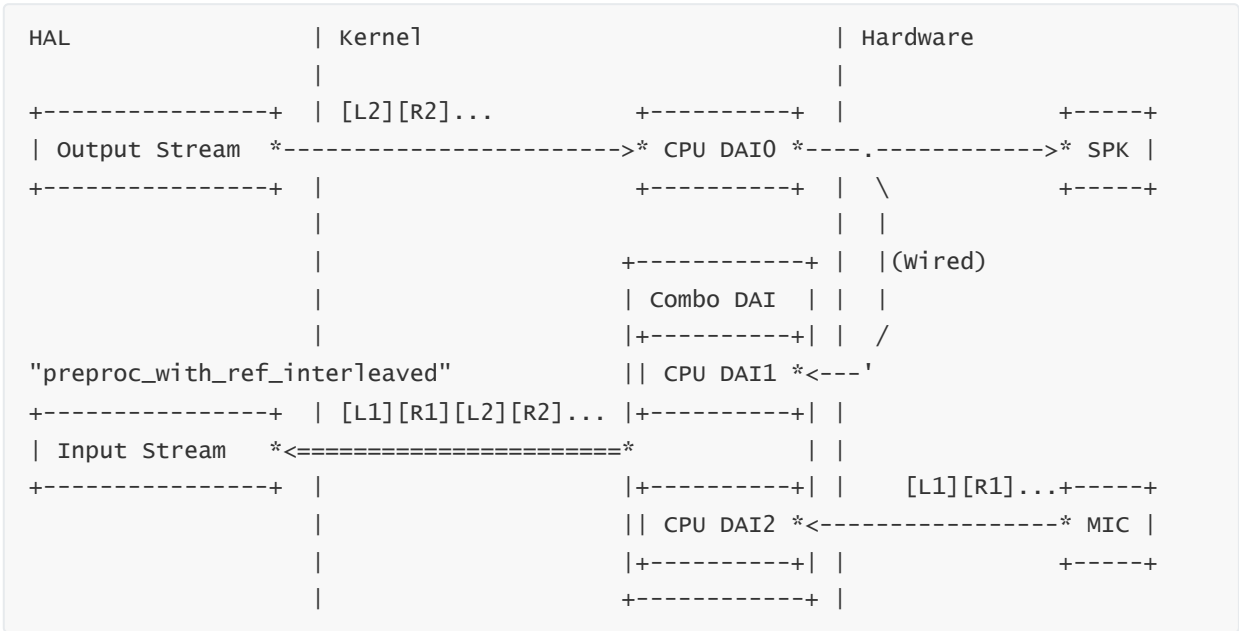
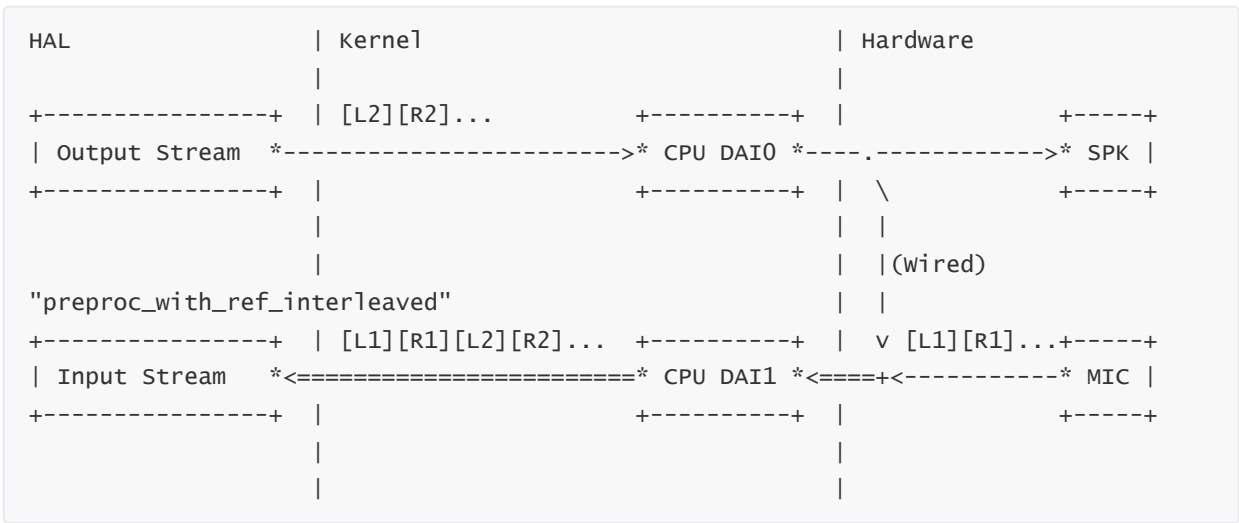
修改 `frameworks/av/media/libeffects/data/audio_effects.xml`

```
<!-- 如果没有声明 preprocess, 则直接添加下面的配置, 并忽略下面的注释 (这是一段注释, 不用粘贴) -->
<preprocess>
  <stream type="mic">
    <!-- 保留已有内容, 添加下面的配置项 (这是一段注释, 不用粘贴) -->
    <apply effect="preproc_with_ref_interleaved"/>
  </stream>
  <stream type="voice_communication">
    <!-- 保留已有内容, 添加下面的配置项 (这是一段注释, 不用粘贴) -->
    <apply effect="preproc_with_ref_interleaved"/>
  </stream>
  <stream type="voice_recognition">
    <!-- 保留已有内容, 添加下面的配置项 (这是一段注释, 不用粘贴) -->
    <apply effect="preproc_with_ref_interleaved"/>
  </stream>
  <stream type="camcorder">
    <!-- 保留已有内容, 添加下面的配置项 (这是一段注释, 不用粘贴) -->
    <apply effect="preproc_with_ref_interleaved"/>
  </stream>
</preprocess>
```

根据项目实际的硬件设计, **参考**以下三种方式之一即可:

1. 硬回采

[L1][R1]... 表示 MIC 数据; [L2][R2]... 表示回采数据, 在 SPK 的前级回环到某个音频接口 (CPU DAI1)



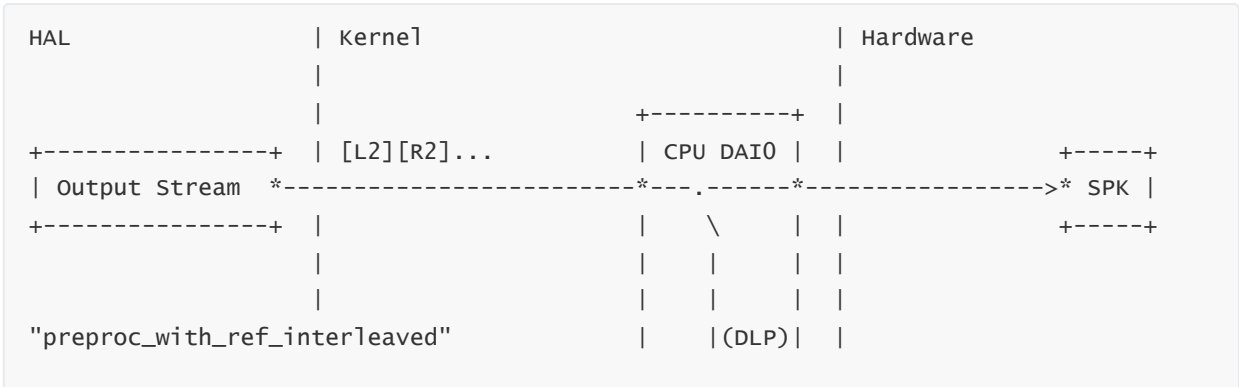
修改回采声道数, 比如 2 channel 回采

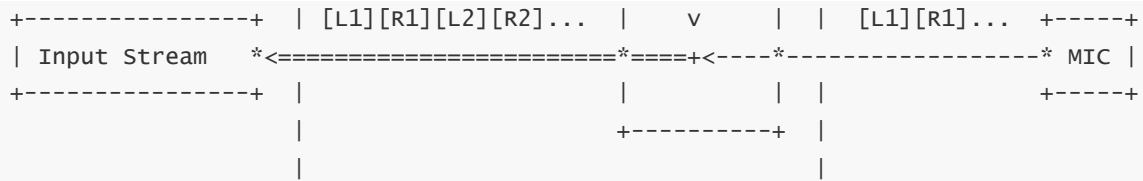
hardware/rockchip/audio/tinyalsa_hal/audio_hw.c

```
#define PCM_REFERENCE_CHANNELS 2
```

2. DLP

[L1][R1]... 表示 MIC 数据; [L2][R2]... 表示回采数据, 缓存到 DLP





修改 kernel 配置, 编译 DLP

```
CONFIG_SND_SOC_ROCKCHIP_DLP_PCM
```

修改 dts 配置, 在对应的 cpu dai 上使能 DLP

```
&i2s {
    // ...
    rockchip,digital-loopback;
    // ...
};
```

cpu dai 可以是: i2s0_8ch, i2s1_8ch, i2s2_2ch, i2s3_2ch, sai0, sai1 等等

修改通路配置, 在 tinyalsa_hal 增加对应的通路配置

不同的声卡名对应不同通路配置文件, 通过 sound_card_config_list 确认, 没有匹配则使用默认的 default_config.h

比如 2 channel mic + 2 channel 回采

```
diff --git a/tinyalsa_hal/codec_config/default_config.h
b/tinyalsa_hal/codec_config/default_config.h
index a554492..114e3d2 100755
--- a/tinyalsa_hal/codec_config/default_config.h
+++ b/tinyalsa_hal/codec_config/default_config.h
@@ -19,6 +19,10 @@ const struct config_control default_speaker_normal_controls[] = {
    .ctl_name = "Playback Path",
    .str_val = "SPK",
},
+ {
+     .ctl_name = "Software Digital Loopback Mode",
+     .str_val = "4CH: 2 Mics + 2 Loopbacks",
+ },
};

@@ -198,6 +202,10 @@ const struct config_control default_main_mic_capture_controls[]
= {
    .ctl_name = "Capture MIC Path",
    .str_val = "Main Mic",
},
+ {
+     .ctl_name = "Software Digital Loopback Mode",
+     .str_val = "4CH: 2 Mics + 2 Loopbacks",
+ },
};
```

```
};

const struct config_control default_hands_free_mic_capture_controls[] = {
```

总共有以下几个选项, 根据对应的 MIC 声道数和回采声道数配置即可

```
static const char *const dlp_text[] = {
    "Disabled",
    "2CH: 1 Loopback + 1 Mic",
    "2CH: 1 Mic + 1 Loopback",
    "2CH: 1 Mic + 1 Loopback-mixed",
    "2CH: 2 Loopbacks",
    "4CH: 2 Mics + 2 Loopbacks",
    "4CH: 2 Mics + 1 Loopback-mixed",
    "4CH: 4 Loopbacks",
    "6CH: 4 Mics + 2 Loopbacks",
    "6CH: 4 Mics + 1 Loopback-mixed",
    "6CH: 6 Loopbacks",
    "8CH: 6 Mics + 2 Loopbacks",
    "8CH: 6 Mics + 1 Loopback-mixed",
    "8CH: 8 Loopbacks",
    "10CH: 8 Mics + 2 Loopbacks",
    "10CH: 8 Mics + 1 Loopback-mixed",
    "16CH: 8 Mics + 8 Loopbacks",
};
```

修改回采声道数, 比如 2 channel 回采

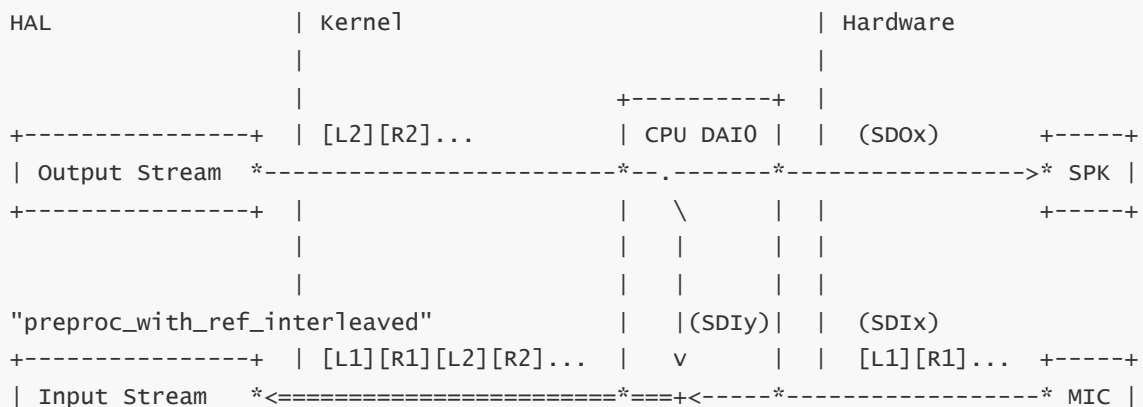
hardware/rockchip/audio/tinyalsa_hal/audio_hw.c

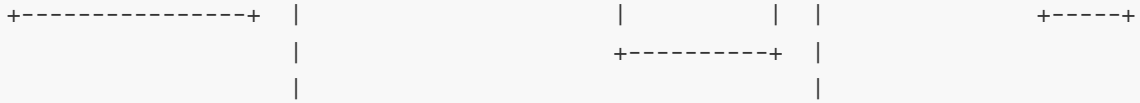
```
#define PCM_REFERENCE_CHANNELS 2
```

3. SAI dataline 间路由

如果 CPU DAI 使用的是 SAI (sai0, sai1 ...), 可以考虑使用 SAI dataline 间路由配置, 回环某个使用的 SDO 到某个空闲的 SDI

[L1][R1]... 表示 MIC 数据; [L2][R2]... 表示回采数据; SDOx 表示正在使用的 SDO; SDIx 表示正在使用的 SDI; SDIy 表示空闲的 SDI, 回环来自 SDOx 的数据





SDI 回环源

sound/soc/rockchip/rockchip_sai.c

```
static const char * const lpx_text[] = {
    "From SDO0", "From SDO1", "From SDO2", "From SDO3" };

static SOC_ENUM_SINGLE_DECL(lp3_enum, SAI_PATH_SEL, 28, lpx_text);
static SOC_ENUM_SINGLE_DECL(lp2_enum, SAI_PATH_SEL, 26, lpx_text);
static SOC_ENUM_SINGLE_DECL(lp1_enum, SAI_PATH_SEL, 24, lpx_text);
static SOC_ENUM_SINGLE_DECL(lp0_enum, SAI_PATH_SEL, 22, lpx_text);

static const struct snd_kcontrol_new rockchip_sai_controls[] = {
    // ...
    SOC_ENUM("SDI3 Loopback Src select", lp3_enum),
    SOC_ENUM("SDI2 Loopback Src select", lp2_enum),
    SOC_ENUM("SDI1 Loopback Src select", lp1_enum),
    SOC_ENUM("SDI0 Loopback Src select", lp0_enum),
    // ...
};
```

SDI 回环开关

sound/soc/rockchip/rockchip_sai.c

```
static const char * const lps_text[] = { "Disable", "Enable" };

static SOC_ENUM_SINGLE_DECL(lp3_switch, SAI_PATH_SEL, 21, lps_text);
static SOC_ENUM_SINGLE_DECL(lp2_switch, SAI_PATH_SEL, 20, lps_text);
static SOC_ENUM_SINGLE_DECL(lp1_switch, SAI_PATH_SEL, 19, lps_text);
static SOC_ENUM_SINGLE_DECL(lp0_switch, SAI_PATH_SEL, 18, lps_text);

static const struct snd_kcontrol_new rockchip_sai_controls[] = {
    // ...
    SOC_ENUM("SDI3 Loopback Switch", lp3_switch),
    SOC_ENUM("SDI2 Loopback Switch", lp2_switch),
    SOC_ENUM("SDI1 Loopback Switch", lp1_switch),
    SOC_ENUM("SDI0 Loopback Switch", lp0_switch),
    // ...
};
```

比如 2 channel mic + 2 channel 回采, 使用 SDO0 输出到 SPK, SDI0 接收来自 MIC 的数据, 可以从空闲的 SDI1, SDI2 和 SDI3 中选择一个回环来自 SDO0 的数据, 比如选择 SDI1 回环来自 SDO0 的数据, 则可以参考下面修改:

修改通路配置, 在 tinyalsa_hal 增加对应的通路配置

不同的声卡名对应不同通路配置文件, 通过 sound_card_config_list 确认, 没有匹配则使用默认的 default_config.h

```
diff --git a/tinyalsa_hal/codec_config/default_config.h
b/tinyalsa_hal/codec_config/default_config.h
index a554492..114e3d2 100755
--- a/tinyalsa_hal/codec_config/default_config.h
+++ b/tinyalsa_hal/codec_config/default_config.h
@@ -198,6 +202,14 @@ const struct config_control default_main_mic_capture_controls[]
= {
    .ctl_name = "Capture MIC Path",
    .str_val = "Main Mic",
},
+ {
+    .ctl_name = "SDI1 Loopback Src Select",
+    .str_val = "From SD00",
+ },
+ {
+    .ctl_name = "SDI1 Loopback Switch",
+    .str_val = "Enable",
+ },
};

const struct config_control default_hands_free_mic_capture_controls[] = {
```

注: 因内核版本差异, 实际的控件名称可能会包含前缀, 如“SAI0 SDI1 Loopback Switch”, 可以通过 tinymix 查看

修改 回采声道数, 比如 2 channel 回采

hardware/rockchip/audio/tinyalsa_hal/audio_hw.c

```
#define PCM_REFERENCE_CHANNELS 2
```

MIC 数据和回采数据异源

异源是指 MIC 数据来源于一个声卡, 而回采数据来源于另一个声卡或 buffer

修改 frameworks/av/media/libeffects/data/audio_effects.xml

```
<!-- 如果没有声明 preprocess, 则直接添加下面的配置, 并忽略下面的注释 (这是一段注释, 不用粘贴) -->
<preprocess>
  <stream type="mic">
    <!-- 保留已有内容, 添加下面的配置项 (这是一段注释, 不用粘贴) -->
    <apply effect="preproc_with_ref_separated"/>
  </stream>
  <stream type="voice_communication">
    <!-- 保留已有内容, 添加下面的配置项 (这是一段注释, 不用粘贴) -->
    <apply effect="preproc_with_ref_separated"/>
  </stream>
```

```

<stream type="voice_recognition">
    <!-- 保留已有内容，添加下面的配置项（这是一段注释，不用粘贴） -->
    <apply effect="preproc_with_ref_separated"/>
</stream>
<stream type="camcorder">
    <!-- 保留已有内容，添加下面的配置项（这是一段注释，不用粘贴） -->
    <apply effect="preproc_with_ref_separated"/>
</stream>
</preprocess>

<!-- 如果没有声明 postprocess，则直接添加下面的配置，并忽略下面的注释（这是一段注释，不用粘贴） -->
<postprocess>
    <stream type="music">
        <!-- 保留已有内容，添加下面的配置项（这是一段注释，不用粘贴） -->
        <apply effect="preproc_ref_cache"/>
    </stream>
    <stream type="voice_call">
        <!-- 保留已有内容，添加下面的配置项（这是一段注释，不用粘贴） -->
        <apply effect="preproc_ref_cache"/>
    </stream>
</postprocess>

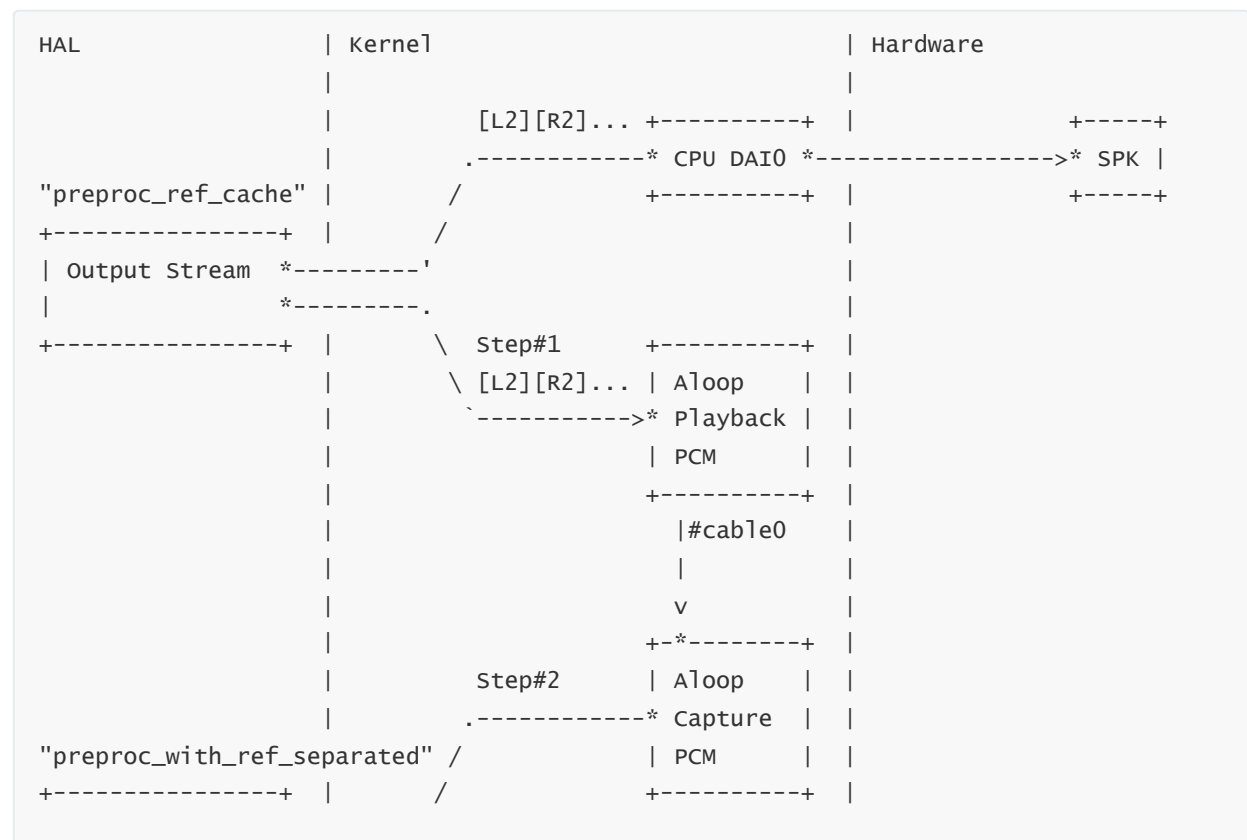
```

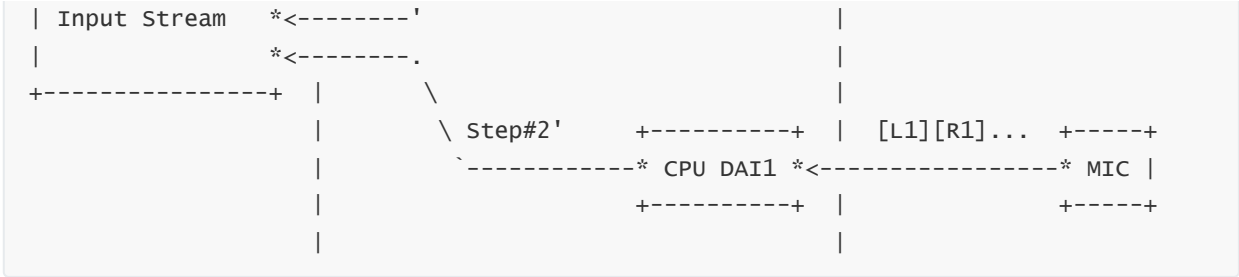
根据实际的支持情况, 参考以下两种方式之一即可:

1. aloop

如下图示: [L1][R1]... 表示 MIC 数据; [L2][R2]... 表示回采数据, 缓存到 aloop

MIC 数据和回采数据最终在 input stream 中拼接





修改 kernel 配置, 编译 aloop

```
CONFIG_SND_DRIVERS
CONFIG_SND_ALOOP
```

修改回采声道数

不需配置回采声道数 (因为 aloop 默认 2 channel)

```
hardware/rockchip/audio/tinyalsa_hal/audio_hw.c
```

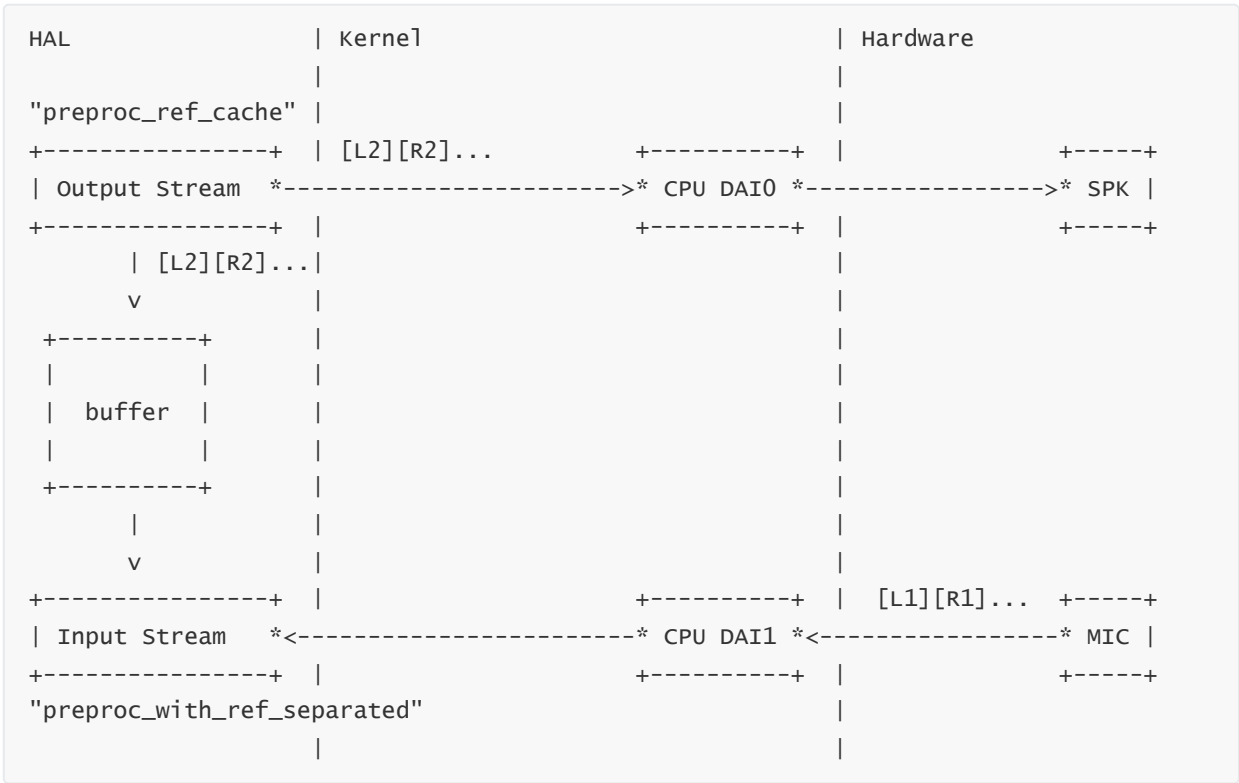
```
#define PCM_REFERENCE_CHANNELS 0
```

确保 PREPROC_ENABLE_ALOOP 已开启, 即使用 aloop 缓存回采数据

```
hardware/rockchip/audio/preprocess/preprocess.c
```

```
#define PREPROC_ENABLE_ALOOP
```

2. buffer



修改回采声道数

不需配置回采声道数 (因为 buffer 默认 2 channel)

hardware/rockchip/audio/tinyalsa_hal/audio_hw.c

```
#define PCM_REFERENCE_CHANNELS 0
```

注释掉 PREPROC_ENABLE_ALOOP 即可使用 buffer 缓存回采数据

hardware/rockchip/audio/preprocess/preprocess.c

```
// #define PREPROC_ENABLE_ALOOP
```

输入仅包含 MIC 数据

在输入仅包含 MIC 数据的情况下, 只需要处理 MIC 数据, 不需要启用 aec/fast_aec/aes

修改 frameworks/av/media/libeffects/data/audio_effects.xml

```
<!-- 如果没有声明 preprocess, 则直接添加下面的配置, 并忽略下面的注释 (这是一段注释, 不用粘贴) -->
<preprocess>
  <stream type="mic">
    <!-- 保留已有内容, 添加下面的配置项 (这是一段注释, 不用粘贴) -->
    <apply effect="preproc_without_ref"/>
  </stream>
  <stream type="voice_communication">
    <!-- 保留已有内容, 添加下面的配置项 (这是一段注释, 不用粘贴) -->
    <apply effect="preproc_without_ref"/>
  </stream>
  <stream type="voice_recognition">
    <!-- 保留已有内容, 添加下面的配置项 (这是一段注释, 不用粘贴) -->
    <apply effect="preproc_without_ref"/>
  </stream>
  <stream type="camcorder">
    <!-- 保留已有内容, 添加下面的配置项 (这是一段注释, 不用粘贴) -->
    <apply effect="preproc_without_ref"/>
  </stream>
</preprocess>
```

调整音频参数

需要设置基本的参数以满足算法库的初始化

修改 MIC 录音参数

需要修改 MIC 录音的帧长为 16ms, 向下取整

```
period_size = (int)(rate * 0.016 / 4) * 4
```

比如采样率 48000, `period_size = (int)(48000 * 0.016 / 4) * 4 = 768`

比如采样率 44100, `period_size = (int)(44100 * 0.016 / 4) * 4 = 704`

确认后**修改**对应的 pcm config, 比如:

`hardware/rockchip/audio/tinyalsa_hal/audio_hw.c`

```
static struct pcm_config pcm_config_in = {
    // ...
    .rate = 48000,
    .period_size = 768,
};
```

确认声道数后**修改**下面的配置, 比如 2 channel MIC

`hardware/rockchip/audio/tinyalsa_hal/audio_hw.c`

```
#define PCM_CAPTURE_CHANNELS 2
```

单 MIC 目前是以 2 channel 来处理, 只取其中一个有效的声道, 忽略另一个声道

关闭 libanr 和 libspeex

libanr 和 libspeex 作为前级会污染算法库的输入, 需要**注释**掉 RK_DENOISE_ENABLE 宏, 将其关闭

`hardware/rockchip/audio/tinyalsa_hal/audio_hw.h`

```
diff --git a/tinyalsa_hal/audio_hw.h b/tinyalsa_hal/audio_hw.h
index dc3987a..2043ab5 100644
--- a/tinyalsa_hal/audio_hw.h
+++ b/tinyalsa_hal/audio_hw.h
@@ -103,7 +103,7 @@
#define MAX_SUPPORTED_SAMPLE_RATES 2

#ifdef BOX_HAL
-#define RK_DENOISE_ENABLE
+// #define RK_DENOISE_ENABLE
#endif

#define HW_PARAMS_FLAG_LPCM 0
```

验证

确认音效是否已经注册到 audioflinger

<!-- 以下所有配置均为示例, 请勿复制 -->

```
<libraries>
  <!-- ... -->
  <library name="rkpreprocess" path="librkpreprocess.so"/>
</libraries>
```

```
<effects>
  <!-- ... -->
  <effect name="preproc_without_ref" library="rkpreprocess" uuid="ca002f38-5304-4e93-93de-0002a5d5c51b"/>
  <effect name="preproc_with_ref_interleaved" library="rkpreprocess" uuid="bc23650c-dddf-47d6-a426-0002a5d5c51b"/>
  <effect name="preproc_with_ref_separated" library="rkpreprocess" uuid="d3d81983-b1fd-459f-99f4-0002a5d5c51b"/>
  <effect name="preproc_ref_cache" library="rkpreprocess" uuid="9adba6e4-ce07-4c50-afbf-0002a5d5c51b"/>
</effects>
```

通过在设备端执行 `dumpsys media.audio_flinger` 确认

```
# dumpsys media.audio_flinger
[...]
Libraries loaded:
[...]
Library rkpreprocess
  path: /vendor/lib/soundfx/librkpreprocess.so
  Reference Cache / Rockchip Electronics Co., Ltd.
    UUID: 9adba6e4-ce07-4c50-afbf-0002a5d5c51b
    TYPE: 34805d32-2e6d-4d1e-9296-0002a5d5c51b
    apiVersion: 00020000
    flags: 00820204
  Preprocess With Reference Separated / Rockchip Electronics Co., Ltd.
    UUID: d3d81983-b1fd-459f-99f4-0002a5d5c51b
    TYPE: 34805d32-2e6d-4d1e-9296-0002a5d5c51b
    apiVersion: 00020000
    flags: 00820203
  Preprocess With Reference Interleaved / Rockchip Electronics Co., Ltd.
    UUID: bc23650c-dddf-47d6-a426-0002a5d5c51b
    TYPE: 34805d32-2e6d-4d1e-9296-0002a5d5c51b
    apiVersion: 00020000
    flags: 00820203
  Preprocess Without Reference / Rockchip Electronics Co., Ltd.
    UUID: ca002f38-5304-4e93-93de-0002a5d5c51b
    TYPE: 34805d32-2e6d-4d1e-9296-0002a5d5c51b
    apiVersion: 00020000
    flags: 00820203
[...]
```

确认音效是否能正常绑定到对应的 stream

以语音, 软回采 aloop 为例

```
<!-- 以下所有配置均为示例，请勿复制 -->

<preprocess>
  <!-- ... -->
  <stream type="voice_communication">
```

```

        <!-- ... -->
        <apply effect="preproc_with_ref_separated"/>
    </stream>
</preprocess>

<postprocess>
    <!-- ... -->
    <stream type="voice_call">
        <!-- ... -->
        <apply effect="preproc_ref_cache"/>
    </stream>
</postprocess>

```

开流后, 通过在设备端执行 `dumpsys media.audio_flinger` 确认

```

# dumpsys media.audio_flinger
[...]
Output thread 0xb40000784395ba60, name AudioOut_D, tid 642, type 0 (MIXER):
  I/O handle: 13
  Standby: no
  [...]
  1 Effect Chains
    1 effects for session 249
      In buffer    Out buffer          Active tracks:
      0x78439e0000 0x7843d34000 -> 0xb400007730317020 1
      Effect ID 75:
        [...]
        - UUID: 9adba6e4-ce07-4c50-afbf-0002a5d5c51b
        [...]
        - name: Reference Cache
        [...]
      [...]
    [...]
  Input thread 0xb40000784388dc60, name AudioIn_2E, tid 6501, type 3 (RECORD):
    I/O handle: 30
    Standby: no
    [...]
    1 Effect Chains
      1 effects for session 257
        In buffer    Out buffer          Active tracks:
        nullptr    nullptr    0
        Effect ID 139:
          [...]
          - UUID: d3d81983-b1fd-459f-99f4-0002a5d5c51b
          [...]
          - name: Preprocess with Reference Separated
          [...]
        [...]
      [...]
    [...]
  [...]

```

确认算法输出是否符合预期

语音或录音后确认效果是否达到预期, 或参考[分析算法输出](#)

算法参数

算法参数有2种配置方式, SDK默认为json 文件的配置方式:

1、默认开启#define ALGO_INIT_BY_CONFIG, 算法库的默认参数配置保存在
rkaudio_effect_preprocess.json

默认保存到 SDK 的 `hardware/rockchip/audio/preprocess/rkaudio_effect_preprocess.json`

出厂默认安装在设备的 `/vendor/etc/rkaudio_effect_preprocess.json`

2、当关闭ALGO_INIT_BY_CONFIG宏的时候, 配置文件为

`hardware/rockchip/audio/preprocess/libs/include/rkaudio_preprocess.h`

算法参数说明

请参考[文档](#)

注: 该文档是按rkaudio_preprocess.h方式进行参数说明, rkaudio_effect_preprocess.json内各模块参数内容是一样的

算法参数调试

确保固件版本是 userdebug, 确保 PC 可以通过 ADB 与设备建立连接

在 PC 端执行以下两条指令:

```
adb root
adb remount
```

初次烧录完固件需要在执行上述两条指令后重启, 再重新执行上述两条指令

更新算法参数

修改 rkaudio_effect_preprocess.json, 并手动 adb push 到设备对应的目录

```
adb push rkaudio_effect_preprocess.json /vendor/etc/
```

分析算法输出

dump 输入和输出的原始数据来确认效果和分析问题

设置为 permissive 模式和开启 rkpreprocess 调试开关, 设备运行期间执行一次即可

```
setenforce 0
setprop vendor.audio.preproc.debug 1
```

重新开始语音或录音一段时间后, 会在 `/data/misc/audioserver/` 目录生成输入和输出文件, 导出并用 [audacity](#) 或其他 PC 工具分析这两个文件

```
adb pull /data/misc/audioserver/preproc_input_xxxhz_xxxch.pcm
adb pull /data/misc/audioserver/preproc_output_xxxhz_xxxch.pcm
```

其中 xxxhz 表示采样率, xxxch 表示声道数

输入文件 表示算法的输入数据流, 而 输出文件 则表示算法的输出

使用 audacity 分析时, 依次点击 文件, 导入, 原始数据, 然后修改采样率和声道数即可

确认符合效果之后把 `rkaudio_effect_preprocess.json` 更新到 `hardware/rockchip/audio/preprocess/`

Q&A

如何确认需要启用哪些算法模块

1. 如果需要回声消除, 确保输入的数据包含回采数据, 需要启用 aec, bf, fast_aec, aes
2. 如果需要降噪, 需要启用 bf, anr
3. 其他需求请参考[文档](#)的介绍并确认

没有生成输出文件

如果算法库初始化失败

```
12-20 09:03:33.753 442 2298 D preproc : preproc_fx_process: failed to initialize
algorithm
12-20 09:03:33.755 442 2298 D preproc-algo: algo_init: failed to initialize
algorithm
```

1. 如果是付费版本, 确认是否授权成功, 参考[授权](#)小节
2. 确认是否正确[修改 MIC 声道数](#)

`hardware/rockchip/audio/tinyalsa_hal/audio_hw.c`

```
#define PCM_CAPTURE_CHANNELS /* 通过实际的硬件设计确认 MIC 声道数 */
```

输出文件数据全 0

1. 确认另一个输入文件的数据是否全 0, 如果也全 0, 确认下面两项
- 确认声卡至 tinyalsa_hal 数据是否正常

```
# tinycap /sdcard/record.wav -D xxx -c xxx -r xxx
```

tinycap 参数根据实际的声卡序号, 声道数以及采样率填上, 之后导出录音文件 record.wav 并分析

- 确认 tinyalsa_hal 至 rkpreprocess 数据是否正常

```
hardware/rockchip/audio/tinyalsa_hal/audio_hw.c
```

```
static int get_next_buffer(/* ... */)
{
    // ...
    if (in->frames_in == 0) {
        // ...
        in->read_status = pcm_read(in->pcm,
                                   (void*)in->buffer, size);

        // 关注 in->buffer
        audio_effect_process(&in->effects, (void *)in->buffer, (void *)in->buffer,
                             in->config->period_size);

        // ...
    }
    // ...
}
```

2. 确认是否正确[修改 MIC 录音帧长](#), 请参考对应的章节重新确认

```
hardware/rockchip/audio/tinyalsa_hal/audio_hw.c
```

```
static struct pcm_config pcm_config_in = {
    // ...
    .rate = /* 采样率 */,
    .period_size = /* 采样率对应的 16ms 帧长 */,
};
```

如果在启用某个算法库的模块时出现这个问题, 则说明这个模块在当前版本的算法库不受支持, 请关闭模块或者申请受支持的算法库版本

加入 DLP 后, 无法以指定的声道 (MIC + 回采) 录音, 提示参数错误

以 2 channel MIC 和 2 channel 回采为例

```
# tinycap /sdcard/mic.wav -D xxx -c 4 -r 48000
Unable to open PCM device (cannot set hw params: Invalid argument)
Captured 0 frames
```

xxx 是 cat /proc/asound/cards 后看到的对应的声卡序号

检查通路设置

1. 确认是否能正常设置通路

```
# tinymix -D xxx "Software Digital Loopback Mode" "4CH: 2 Mics + 2 Loopbacks"
```

如果报错了需要确以下 3 点

```
Invalid mixer control: Software Digital Loopback Mode
```

- 需要确认当前版本的 kernel 是否增加了 DLP 支持, 没有则更新 SDK 或加入补丁

```
# grep rockchip,digital-loopback -r ./sound/soc/rockchip/
./sound/soc/rockchip/rockchip_sai.c:    if (device_property_read_bool(&pdev->dev,
"rockchip,digital-loopback"))
./sound/soc/rockchip/rockchip_i2s.c:    if (device_property_read_bool(&pdev->dev,
"rockchip,digital-loopback"))
./sound/soc/rockchip/rockchip_i2s_tdm.c:        if (device_property_read_bool(dev,
"rockchip,digital-loopback")) {
```

- 需要确认 kernel 是否开启了 DLP

```
CONFIG_SND_SOC_ROCKCHIP_DLP_PCM
```

- 需要在 dts 对应的 cpu dai 上使能 DLP

```
&i2s {
    rockchip,digital-loopback;
};
```

cpu dai 可以是: i2s0_8ch, i2s1_8ch, i2s2_2ch, i2s3_2ch, sai0, sai1 等等

2. 确认通路是否生效

```
# tinymix -D xxx
...
13      ENUM      1      Software Digital Loopback Mode      4CH: 2 Mics + 2
Loopbacks
...
```

检查声卡相关

确认 dts 绑定的 codec dai driver 是否声明支持 4 channel capture

```
// ...
demo_sound: demo-sound {
    compatible = "simple-audio-card";
    // ...
    simple-audio-card,codec {
        sound-dai = <&demo>;
    };
    // ...
};

// ...
// 或者
demo_sound: demo-sound {
    compatible = "rockchip,multicodecs-card";
    // ...
    rockchip,codec = <&demo>;
```

```
// ...  
};
```

```
struct snd_soc_dai_driver demo_dai = {  
    // 没有 capture, 或者 channels_max 小于 4 channel, 则说明不支持  
    .capture = {  
        .channels_min = 1,  
        .channels_max = 2,  
    },  
    // ...  
};
```

如果不支持则可以使用 "rockchip,multicodecs-card", 同时绑定一个辅助的虚拟 codec

```
// 修改前 (例1)  
#if 0  
demo_sound: demo-sound {  
    status = "okay";  
    compatible = "simple-audio-card";  
    simple-audio-card,format = "i2s";  
    simple-audio-card,name = "rockchip,demo-codec";  
    simple-audio-card,mclk-fs = <256>;  
    simple-audio-card,cpu {  
        sound-dai = <&i2s>;  
    };  
    simple-audio-card,codec {  
        sound-dai = <&demo>;  
    };  
};  
#endif
```

```
// 修改前 (例2)  
#if 0  
demo_sound: demo-sound {  
    status = "okay";  
    compatible = "rockchip,multicodecs-card";  
    rockchip,format = "i2s";  
    rockchip,card-name = "rockchip,demo-codec";  
    rockchip,mclk-fs = <256>;  
    rockchip,cpu = <&i2s>;  
    rockchip,codec = <&demo>;  
};  
#endif
```

```
// 修改后  
// 声明虚拟 codec 节点  
demo_aux: demo-aux {  
    status = "okay";  
    compatible = "rockchip,dummy-codec";  
    sound-name-prefix = "demo-aux";
```

```
#sound-dai-cells = <0>;  
};  
  
// 使用 "rockchip,multicodecs-card" 绑定 codec 和虚拟 codec  
demo_sound: demo-sound {  
    status = "okay";  
    compatible = "rockchip,multicodecs-card";  
    rockchip,format = "i2s";  
    rockchip,card-name = "rockchip,demo-codec";  
    rockchip,mclk-fs = <256>;  
    rockchip,cpu = <&i2s>;  
    rockchip,codec = <&demo &demo_aux>;  
};
```